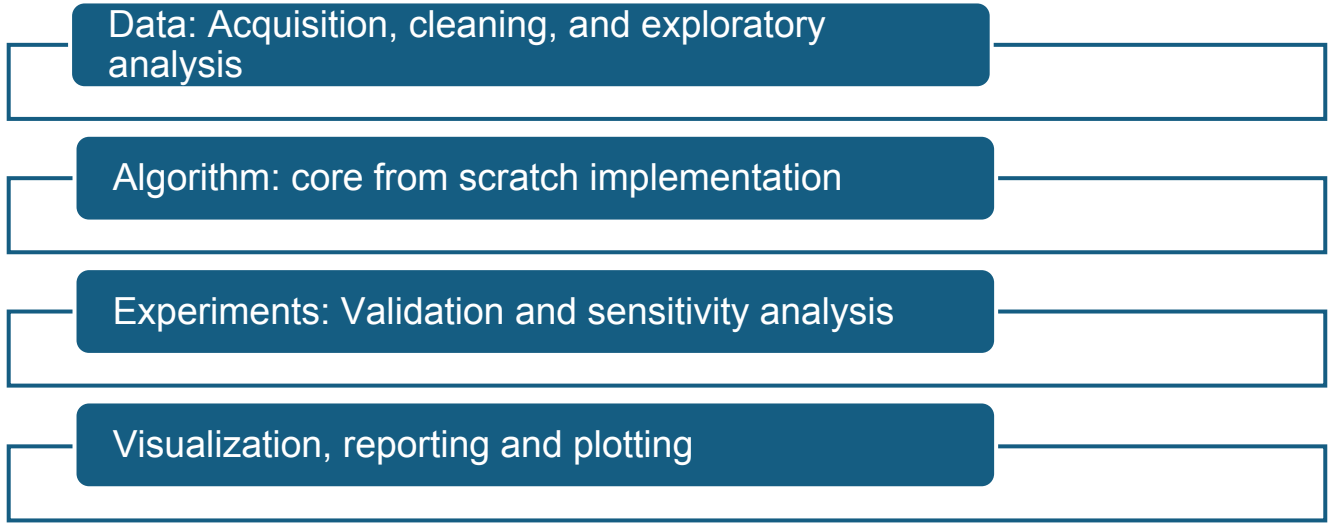
Data Science Tools & Techniques Lab

Project Allocation and Guidelines

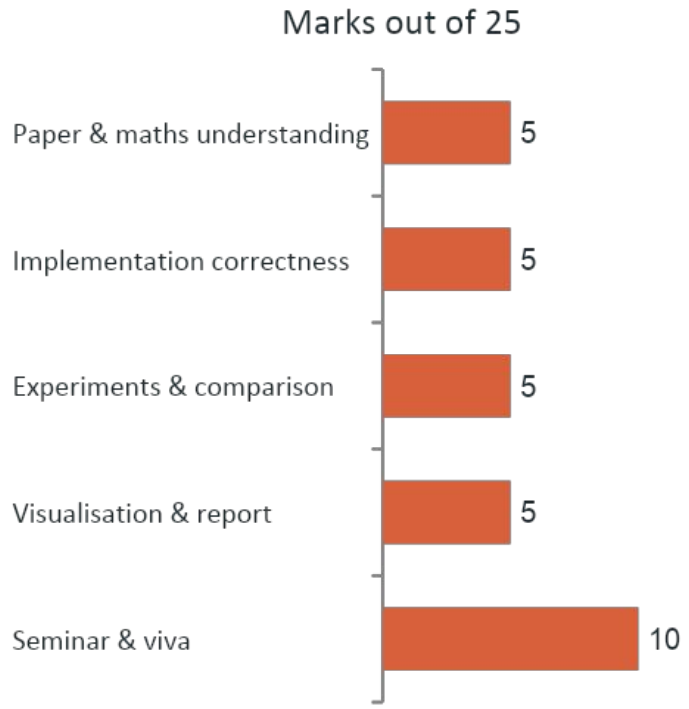
Project guidelines

- Each group of 4 students shall be assigned a project.
- Project work: Read the selected research paper, implement its core algorithm yourself, test it on real data, and explain your algorithm and results in the seminar. Submit the project in google classroom for evaluation.



Deliverables and Evaluation

- A Jupyter notebook that runs top to bottom without errors.
- 4-6 pages: paper summary, mathematics, implementation, results, limitations.
- 15-minute talk followed by 5 minutes of Q&A



Data Wrangling and Mess

- Real data arrives as text, CSV, JSON and BSON exports from different systems. They run on different clocks, use different units, sample at different rates, and contain gaps, duplicates and faulty sensors. A model trained on that is only as good as the cleaning behind it.
- JavaScript Object Notation (JSON) is a standard text-based format for representing structured data based on JavaScript object syntax. It is commonly used for transmitting data in web applications (e.g., sending some data from the server to the client, so it can be displayed on a web page, or vice versa).
- You have to access array items (in its parsed version) by starting with an array index, for example superHeroes[0].powers[0].
- The JSON can also contain a single primitive. For example, 29, "Dan Jukes", or true are all valid JSON.


```
[
  {
    "name": "Molecule Man",
    "age": 29,
    "secretIdentity": "Dan Jukes",
    "powers": ["Radiation resistance", "Turning tiny
"Radiation blast"]
  },
  {
    "name": "Madame Uppercut",
    "age": 39,
    "secretIdentity": "Jane Wilson",
    "powers": [
      "Million tonne punch",
      "Damage resistance",
      "Superhuman reflexes"
    ]
  }
]
```

BSON

BSON stands for Binary Javascript Object Notation. It is a binary-encoded serialization of JSON documents. BSON has been extended to add some optional non-JSON-native data types, like dates and binary data.

```
import bson
# 1. Define your data

data = { "name": "Alice", "age": 30, "skills": ["Python", "MongoDB"] }

# 2. Serialize and write to a .bson file
with open("data.bson", "wb") as file:
file.write(bson.encode(data))
```

```
!pip install chdb

from chdb.datastore import DataStore
df = DataStore.from_file("data/events.bson", format="BSONEachRow")
print(df)
print(df.dtypes)


pdf = df.to_pandas()
```

```
event_id country event_type  revenue
0         0      GB   purchase    37.41
1         1      AU       view    76.47
2         2      IN       cart    67.61
3         3      GB   purchase    87.69
4         4      AU       view    18.12
5         5      IN       cart    35.58
event_id        int64
country           str
event_type        str
revenue       float64
dtype: object
```

Many formats: TXT, CSV, JSON, BSON, fixed-width, candump

Schema drift: columns renamed or reordered after upgrades

Clocks & time zones: UTC vs IST, epochs, wrong device clocks

Mixed frequencies: 0.1 Hz to hourly, irregular samples

Units: psi/bar, mph, °F, mg/m³, ppb, ₹ strings

Duplicates & overlaps: repeated exports, pagination, retries

Faulty values: sentinels, stuck sensors, spikes, gaps

Identity: same entity, different IDs or codes

Recovering a Week of Pump-Station Sensor Data

TXT	**plc_A/** bearing temperature, run status
CSV	**vibration/** RMS and peak vibration
CSV	**scada/** pressure and flow (long format)
JSON	**events/** start/stop/maintenance
BSON	**cmms/** calibration records

pump_1min.csv: 10,080 one-minute rows, 6 signals in SI units, IST

Questions to investigate

- Several clocks and units in one week
- Layouts that change mid-week
- Values that repeat for hours
- A maintenance note that changes the numbers

Anomaly detection. A real developing fault is hidden in the data: keep it.

Decoding and Repairing Car OBD-II / CAN Logs

candump	**can_logs/** raw hex CAN frames, 7 files
CSV	**phone_app/** two app versions, 3 trips
JSON	**meta/** trip list, GPS time fixes

You deliver

trips_1hz.csv (9 trips, 7 signals, true UTC) + trip_features.csv

Questions to investigate

- Decoding bytes with the SAE J1979 formulas
- A logger with no real-time clock
- A trip split across two files
- Imperial vs metric app exports

Driving-style clustering: distance, idle share, harsh accelerations, fuel used.

Decoding and Repairing Car OBD-II / CAN Logs

candump	**can_logs/** raw hex CAN frames, 7 files
CSV	**phone_app/** two app versions, 3 trips
JSON	**meta/** trip list, GPS time fixes

You deliver

trips_1hz.csv (9 trips, 7 signals, true UTC) + trip_features.csv

Questions to investigate

- Decoding bytes with the SAE J1979 formulas
- A logger with no real-time clock
- A trip split across two files
- Imperial vs metric app exports

Driving-style clustering: distance, idle share, harsh accelerations, fuel used.

Merging a City Air-Quality Network

CSV	**S01_S02_portal/** portal export with a header block
fixed-width	**S03_legacy/** old hourly logger
JSON	**S04_api/** daily API pages
BSON	**S05_mongo/** MongoDB dump

You deliver

aq_hourly.csv (5 stations × 1,080 h) + aq_features.csv

Questions to investigate

- What "hour 24" means
- Time zones stated only in a header
- Old units and physically impossible values
- City-wide events vs broken sensors

Next-hour PM2.5 forecasting with lag features, without leaking the future.

Consolidating Retail Sales and CRM Data

CSV	**branch_A/** monthly, POS upgrade mid-way
TXT	**branch_B/** semicolon, legacy customer codes
TSV	**branch_C/** one dump, rupee strings
JSON	**catalog/** products, dirty categories
BSON	**crm/** customers with duplicates

You deliver

sales_clean.csv + customer_map.csv + customer_features.csv

Questions to investigate

- Is 03/04 March or April?
- Files exported twice
- Test and voided transactions
- One person, two customer IDs

Churn prediction with a proper observation / label time split.

Same Statistics, Different Graphs

What to implement

1. Compute mean, variance, Pearson correlation and the least-squares line for Anscombe's quartet using NumPy formulas only.
2. Show all four datasets share near-identical statistics, then plot them to reveal how different they are.
3. Repeat the analysis with the Datasaurus Dozen.
4. Implement a simplified simulated-annealing version: nudge points toward a target shape while keeping statistics fixed to two decimals.

Paper

Anscombe, F. J. (1973). Graphs in Statistical Analysis. The American Statistician, 27(1), 17-21.

Matejka, J. & Fitzmaurice, G. (2017). Same Stats, Different Graphs. Proc. CHI 2017.

Dataset

Anscombe / seaborn-data
github.com/mwaskom/seaborn-data

Datasaurus Dozen
www.autodesk.com/research/publications/same-stats-different-graphs

Benford's Law and Fraud Detection

Probability distributions, goodness-of-fit

What to implement

1. Derive first-digit probabilities $P(d) = \log10(1 + 1/d)$.
2. Extract leading digits from real datasets using pandas.
3. Implement the chi-square goodness-of-fit test and the MAD conformity test from scratch.
4. Generate "fabricated" data (e.g. uniform random numbers) and show that it fails the test.
5. Discuss why some data, such as human heights, should not follow Benford's law.

Paper

Benford, F. (1938). The Law of Anomalous Numbers. Proceedings of the American Philosophical Society, 78(4), 551-572.

Dataset

World Bank population
data.worldbank.org/indicator/SP.POP.TOTL

Bootstrap and Jackknife Confidence Intervals

What to implement

1. Implement the nonparametric bootstrap and the jackknife for the mean, median and correlation coefficient.
2. Build percentile confidence intervals and compare them with the classical t-interval.
3. Study how interval width changes with sample size and number of replicates B.
4. Run a simulation to check whether a 95% interval covers the true value about 95% of the time.

Paper

Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. Annals of Statistics, 7(1), 1-26.

Dataset

Wine Quality (UCI)
archive.ics.uci.edu/dataset/186/wine+quality

Ridge Regression and Multicollinearity

What to implement

1. Solve OLS via the normal equations, then via QR decomposition and SVD; compare.
2. Compute the condition number of X^TX and show how multicollinearity destabilises coefficients.
3. Implement the ridge estimator $(X^TX + \lambda I)^{-1}X^Ty$ and reproduce the "ridge trace" plot.
4. Choose λ with k-fold cross-validation written in NumPy and pandas.

Paper

Hoerl, A. E. & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. Technometrics, 12(1), 55-67.

Dataset

Longley data
statsmodels.datasets.longley

Real Estate Valuation (UCI)
archive.ics.uci.edu/dataset/477/real+estate+valuation+data+set

Logistic Regression: Gradient Descent vs Newton

What to implement

1. Derive the log-likelihood, its gradient and Hessian by hand.
2. Implement batch GD, stochastic GD and Newton-Raphson (IRLS).
3. Compare convergence by plotting loss against iteration.
4. Evaluate with accuracy, confusion matrix and ROC curve, all from scratch.
5. Show what happens when the learning rate is too high.

Paper

Cox, D. R. (1958). The Regression Analysis of Binary Sequences. Journal of the Royal Statistical Society B, 20(2), 215-242.

Dataset

Breast Cancer Wisconsin (UCI)
archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

Eigenfaces: PCA for Face Recognition

What to implement

1. Implement PCA through eigen-decomposition of the covariance matrix.
2. Implement PCA via SVD, using the paper's AA^T vs A^TA "small matrix" trick.
3. Visualise the mean face and top eigenfaces; plot explained variance against k.
4. Reconstruct faces using k components.
5. Build a nearest-neighbour recogniser in eigenface space; report accuracy against k.

Paper

Turk, M. & Pentland, A. (1991). Eigenfaces for Recognition. Journal of Cognitive Neuroscience, 3(1), 71-86.

Dataset

Olivetti faces (400 images)
sklearn.datasets.fetch_olivetti_faces

PageRank on Real Networks

What to implement

1. Build the column-stochastic transition matrix from an edge list with pandas.
2. Handle dangling nodes and add the damping factor.
3. Solve by power iteration and by the eigenvector of the Google matrix; compare.
4. Study how the damping factor affects convergence and ranking.
5. Compare PageRank with a simple in-degree ranking.

Paper

Page, L., Brin, S., Motwani, R. & Winograd, T. (1999). The PageRank Citation Ranking: Bringing Order to the Web. Stanford InfoLab Technical Report.

Dataset

Wikipedia vote network (SNAP)
snap.stanford.edu/data/wiki-Vote.html

Email network (SNAP)
snap.stanford.edu/data/email-Eu-core.html

k-Means and k-Means++ Customer Segmentation

What to implement

1. Compute Recency, Frequency and Monetary (RFM) values per customer with pandas.
2. Implement Lloyd's algorithm with vectorised NumPy distances.
3. Implement k-means++ seeding and compare with random initialisation over many runs.
4. Choose k with the elbow method and silhouette score, both from scratch.
5. Interpret each segment in business terms.

Paper

Lloyd, S. (1982). Least Squares Quantization in PCM. IEEE Trans. Information Theory, 28(2), 129-137.

Arthur, D. & Vassilvitskii, S. (2007). k-means++: The Advantages of Careful Seeding. Proc. SODA.

Dataset

Online Retail (UCI)
archive.ics.uci.edu/dataset/352/online+retail

EM Algorithm for Gaussian Mixtures

What to implement

1. Implement the multivariate Gaussian density in NumPy.
2. Implement the E-step (responsibilities) and M-step (means, covariances, weights).
3. Verify that the log-likelihood never decreases.
4. Plot or animate the fitted clusters as they evolve.
5. Compare GMM soft assignments with k-means hard assignments.

Paper

Dempster, A. P., Laird, N. M. & Rubin, D. B. (1977). Maximum Likelihood from Incomplete Data via the EM Algorithm. JRSS B, 39(1), 1-38.

Dataset

Old Faithful (geyser.csv)
github.com/mwaskom/seaborn-data

Iris (UCI)
archive.ics.uci.edu/dataset/53/iris

Savitzky-Golay Smoothing and Differentiation

What to implement

1. Derive convolution coefficients by local least-squares polynomial fitting (Vandermonde matrix).
2. Implement smoothing and first and second derivatives.
3. Compare with moving average, naive finite differences and scipy.signal.savgol_filter.
4. Apply to noisy real data: growth rates (1st derivative) and turning points (2nd derivative).

Paper

Savitzky, A. & Golay, M. J. E. (1964). Smoothing and Differentiation of Data by Simplified Least Squares Procedures. Analytical Chemistry, 36(8), 1627-1639.

Dataset

NOAA Mauna Loa CO_2
gml.noaa.gov/ccgg/trends/data.html

Our World in Data COVID-19
github.com/owid/covid-19-data

FFT and Hidden Cycles in Data

What to implement

1. Implement the naive $O(n^2)$ DFT and the recursive radix-2 FFT.
2. Verify against np.fft.fft and benchmark run time as n grows.
3. Recover the ~11-year solar cycle from the sunspot power spectrum.
4. Find daily and weekly cycles in electricity consumption.
5. Demonstrate aliasing and zero-padding.

Paper

Cooley, J. W. & Tukey, J. W. (1965). An Algorithm for the Machine Calculation of Complex Fourier Series. Mathematics of Computation, 19(90), 297-301.

Dataset

SILSO sunspot numbers
www.sidc.be/SILSO/datafiles

Household Power Consumption (UCI)
archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption

SIR Epidemic Model with ODE Solvers

What to implement

1. Implement Euler and RK4 for the SIR equations; study error against step size.
2. Fit β and γ to real data by minimising squared error (grid search or gradient descent).
3. Estimate R_0 and explain "flattening the curve".
4. Bonus: digitise the paper's 1905-06 Bombay plague data and reproduce the fit.

Paper

Kermack, W. O. & McKendrick, A. G. (1927). A Contribution to the Mathematical Theory of Epidemics. Proc. Royal Society A, 115(772), 700-721.

Dataset

Our World in Data COVID-19
github.com/owid/covid-19-data

Kalman Filter for Noisy Sensor Data

What to implement

1. Implement predict and update equations in matrix form; start with a 1-D estimator.
2. Build a 2-D constant-velocity tracker on synthetic noisy positions; show covariance shrinking.
3. Apply to real sensor readings with gaps and noise.
4. Compare with moving average and exponential smoothing.
5. Study the effect of process- and measurement-noise settings.

Paper

Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering, 82(1), 35-45.

Dataset

Air Quality (UCI)
archive.ics.uci.edu/dataset/360/air+quality

Synthetic trajectory
generated with NumPy

TF-IDF and Latent Semantic Analysis

What to implement

1. Tokenise text and build a term-document matrix with pandas.
2. Implement TF-IDF weighting and cosine similarity from scratch.
3. Build a mini search engine returning the top-k documents for a query.
4. Apply truncated SVD (LSA) and show synonym-aware retrieval; plot documents in 2-D.
5. Bonus: use TF-IDF features to separate spam from ham.

Paper

Salton, G. & Buckley, C. (1988). Term-Weighting Approaches in Automatic Text Retrieval. Information Processing & Management, 24(5), 513-523.

Deerwester, S. et al. (1990). Indexing by Latent Semantic Analysis. JASIS, 41(6), 391-407.

Dataset

SMS Spam Collection (UCI)
archive.ics.uci.edu/dataset/228/sms+spam+collection

Project Gutenberg
www.gutenberg.org

Neural Network from Scratch with Backpropagation

What to implement

1. Implement a two-layer network (sigmoid/ReLU hidden, softmax output) in NumPy.
2. Derive backprop gradients by hand; verify with numerical gradient checking.
3. Train with mini-batch gradient descent.
4. Plot training/validation loss; visualise learned hidden weights as images.
5. Reproduce the XOR problem to show why a hidden layer is needed.

Paper

Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning Representations by Back-Propagating Errors. Nature, 323, 533-536.

Dataset

Optical Handwritten Digits (UCI)
archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits